Tower Zeta6 Slurm Service

Complete User Manual

Edition: 1.2
Verified: 11 November 2026
Service contact: `b.y.ding@tudelft.nl`

Contents

1. Purpose and scope
2. Service model
3. Account application and approval
4. Portal password and SSH keys
5. Tailscale and SSH access
6. Storage and file transfer
7. Modules and Python environments
8. Slurm fundamentals
9. Batch jobs
10. Monitoring and cancellation
11. Usage, quotas and warnings
12. Reproducibility and data safety
13. Troubleshooting
14. Security and acceptable use
15. Support checklist
16. Appendix A — Runbook sequence
17. Appendix B — Command summary

1. Purpose and scope

Tower Zeta6 is a controlled, single-node Slurm service for approved research and development, primarily Python and CUDA deep-learning workloads. This manual covers account application, Portal use, SSH access, storage, Python environments, interactive and batch jobs, monitoring, troubleshooting and acceptable use.

The Quick Start Guide is the shortest supported onboarding path. The ordered runbooks contain copyable commands and templates. This manual is authoritative when a runbook needs policy or architectural context.

2. Service model

2.1 Access path

Users connect to `z6-login`, an isolated Ubuntu login VM. It supports editing, file transfer, environment creation and Slurm commands, but has no GPU. Jobs submitted from it execute on the physical compute node `tower-zeta6`.

The supported path is:

1. Tailscale provides private network reachability.
2. The Portal manages applications, web passwords and SSH public keys.
3. OpenSSH authenticates a personal Linux account by public key.
4. Slurm allocates CPU, memory and GPU resources on the compute node.

The Portal password is not a Linux password. Linux password authentication is disabled, and the service never emails a plaintext SSH password.

2.2 Compute resources

The default `gpu` partition currently contains one compute node with:

- 66 schedulable logical CPUs;
- 249,000 MB configured Slurm memory; and
- two NVIDIA RTX PRO 6000 Blackwell Max-Q GPUs.

Host and login services reserve additional CPU and memory that are not available to jobs. Request only what the workload needs. Resource availability and policy may change; `sinfo` and `scontrol show node tower-zeta6` show current scheduler data.

3. Account application and approval

You need an approved application, active Portal account, registered SSH public key and a Tailscale share for `z6-login`.

While connected to the invited Tailscale account, open:

```
https://z6-login.tail8da8d4.ts.net/apply/
```

Provide a personal Linux username, full name, email, research purpose, estimated annual GPU hours and required persistent storage. Estimates support capacity planning. They do not guarantee immediate scheduling priority. The administrator separately assigns a maximum concurrent GPU count, normally one. This control is not part of the application form and appears on the Portal Account page after approval.

Application states include:

- **Pending:** waiting for administrative review.
- **Queued for provisioning / Provisioning:** approved, but host work remains.
- **Provisioned:** Linux account, storage and key synchronization are ready.
- **Provisioning failed:** an administrator must correct the recorded problem.
- **Rejected:** no account will be created for this request.

Do not attempt SSH until the status is **Provisioned**. Follow Runbook 01 for the exact sequence.

4. Portal password and SSH keys

Approval sends a one-time activation link. Open it, choose a unique Portal password and sign in. If it does not arrive, check spam before requesting a resend.

- Change a known password at `/password/change/`.

- Reset a forgotten password at `/password/reset/`.
- Changing the Portal password does not change SSH authentication.

Create a dedicated Ed25519 key with a strong passphrase. Register only the single-line `.pub` content under **Account → Add key**. Give each device a recognizable name. Key changes normally synchronize within one minute.

Revoke a lost or retired device immediately. Never upload, email or paste the private key file. Detailed commands are in Runbook 02 and Runbook 03.

5. Tailscale and SSH access

Install Tailscale locally and sign in with the exact identity invited by the administrator. Configure this SSH entry, replacing the username:

```
Host z6
    HostName z6-login.tail8da8d4.ts.net
    Port 22
    User YOUR_LINUX_USERNAME
    IdentityFile ~/.ssh/z6_ed25519
    IdentitiesOnly yes
```

Connect with `ssh z6`. On first contact, verify the ED25519 host-key fingerprint out of band:

```
SHA256:RPjy+qUOX/92xw55a8WmcaY2cXV4x1gbtUcs6mqILH0
```

Stop if SSH later reports a changed key. Do not delete the stored key merely to silence the warning. Landing on `z6-login` is correct; users must not connect directly to the physical compute host.

6. Storage and file transfer

6.1 Storage locations

`/home/<user>` is NVMe-backed storage for code, virtual environments and small active files. A normal approved home hard quota is 256 GiB.

`/projects/<project>` is persistent HDD-backed storage for datasets, checkpoints and large outputs. It is also linked at `~/projects/<project>`. The Portal Account page shows the approved project path and quota.

Neither location is a backup. `/projects` is a single disk. Maintain an approved off-host copy of anything that cannot be recreated.

6.2 Inspect and transfer

```
df -h "$HOME" /projects
du -sh "$HOME" ~/projects/* 2>/dev/null
```

```
scp local-file z6:~/
rsync -av --progress local-directory/ z6:~/projects/YOUR_PROJECT/local-directory/
```

Avoid `rsync --delete` until both paths have been inspected. Follow Runbook 04 for placement and transfer examples.

7. Modules and Python environments

7.1 Lmod

Interactive login shells initialize Lmod. Batch scripts must initialize it explicitly:

```
source /etc/profile.d/lmod.sh
module purge
module load 2026
module avail
module list
```

The `2026` module selects the site module tree. As of this edition, it does not itself provide Python, CUDA, cuDNN or PyTorch. Do not guess module names from Snellius or another cluster. When central modules are published, use `module spider NAME` and the administrator's documented versions.

7.2 Virtual environments

The current supported self-managed baseline is system Python 3.12 in a virtual environment created on `z6-login`:

```
mkdir -p ~/venvs
python3 -m venv ~/venvs/z6-pytorch
source ~/venvs/z6-pytorch/bin/activate
python -m pip install --upgrade pip setuptools wheel
```

Prefer a reviewed project requirements or lock file. For a new PyTorch environment, the current documented baseline is the official PyTorch 2.11.0 CUDA 12.8 wheel set shown in Runbook 05.

The login VM has no GPU. Test `torch.cuda.is_available()` only inside a Slurm GPU allocation. Record `pip freeze`, module output and the Git commit used by each experiment.

8. Slurm fundamentals

8.1 Allocations

All substantial CPU, memory and GPU work must run through Slurm. A request normally specifies:

- `--partition=gpu`;
- `--time=HH:MM:SS`;
- `--gpus=N` for GPU work;
- `--ntasks` and `--cpus-per-task`;

- `--mem` or `--mem-per-cpu`; and
- unique `%x` / `%j` log paths.

The default memory is 3,000 MB per allocated CPU, but explicit realistic memory is preferred. There is no configured maximum wall time; always provide one and discuss runs expected to exceed 24 hours.

A job may not request more GPUs than the maximum concurrent GPU count shown in the Portal. Slurm rejects such a job immediately with a message stating the approved limit. If running jobs already consume that limit, additional valid GPU jobs are accepted and remain pending until capacity is released.

8.2 Scheduling order

Z6 uses Slurm's backfill scheduler. Current multifactor priority weights are zero, so jobs of equal effective priority normally retain submission order, but the queue is not a strict first-in-first-out line. A smaller later job may start when it fits unused resources without delaying an earlier reservation. Pending reasons shown by `squeue` are the reliable explanation for a particular job. `AssocGrpGRES` means this account's running jobs currently consume its approved concurrent GPU allocation.

8.3 Interactive work

Use short `srun --pty bash` allocations for debugging and exit when finished. The exact CPU and GPU forms are in Runbook 06.

9. Batch jobs

Create the log directory before submission:

```
mkdir -p jobs logs
sbatch jobs/my-job.sbatch
```

`#SBATCH` directives are read only before the first executable shell command. Shell variables are not expanded inside those directives.

9.1 Single GPU

The supported example requests one GPU, performs a CUDA preflight and runs a small PyTorch calculation. Copy the single-GPU template, then replace only its final smoke-test command with the project entry point.

9.2 Two GPUs

For ordinary single-node PyTorch Distributed, request two GPUs and let one Slurm task start two workers with `torchrun --nproc-per-node=2`. Use the two-GPU template. This template is available only to accounts whose Portal allocation shows two maximum concurrent GPUs; the normal default is one.

Framework-specific launchers may use a different task model. Confirm their documentation instead of mixing a Snellius template with the Z6 resource profile. `--exclusive` is not required merely because both GPUs are requested.

9.3 Arrays

Arrays are appropriate for independent parameter values. Limit concurrency to real capacity, for example `#SBATCH --array=0-3%2` for at most two simultaneous one-GPU elements. See Runbook 09.

10. Monitoring and cancellation

```
squeue -u "$USER"
gpustat
gpustat -i 2
scontrol show job JOB_ID
squeue -j JOB_ID -o "%.18i %.9P %.8T %.30R"
tail -f logs/JOB_NAME-JOB_ID.out
scancel JOB_ID
```

`gpustat` on `z6-login` (and the **GPU status** page at `/status/` in the Portal) reports live GPU utilization, VRAM usage, power draw, temperature and active processes **only for GPUs on which your own Slurm job is currently** `RUNNING`. Use `gpustat -i 2` to refresh every two seconds or `gpustat --json` for JSON output. When you have no running GPU job, live GPU telemetry is not shown.

`Resources` normally means requested hardware is occupied. `Priority` normally means the scheduler is holding the job behind other work. Cancel broken, idle or unnecessary work promptly.

Completed jobs disappear from `squeue`. Inspect historical records with `sacct -j JOB_ID --format=JobID,State,Elapsed,AllocTRES`; durable stdout/stderr and experiment metadata remain essential. See Runbook 10.

11. Usage, quotas and warnings

The Portal Account page displays approved annual GPU hours, maximum concurrent GPUs, storage request, home quota, project quota and project path. These values describe the approved allocation; the current user page does not yet visualise historical job accounting.

Slurm records job accounting, but annual GPU hours remain planning information and are not an automatic annual spending cap. Users remain responsible for honest requests and coordination. Filesystem hard quotas are enforced. A write that exceeds quota fails even if the Portal still shows the approved amount rather than live byte usage.

Use `du` and `df` regularly. Reduce caches, duplicate datasets and obsolete checkpoints before requesting an allocation change.

12. Reproducibility and data safety

For each experiment retain:

- Slurm job ID and batch script;
- Git commit and configuration;

- dataset version;
- module list and Python dependency lock;
- stdout/stderr logs; and
- enough checkpoints to resume a long run.

Submit from a stable project directory. Avoid many workers writing large numbers of small files to `/projects`. Stage suitable small active files in home within quota, then copy durable results to project storage and an off-host backup.

13. Troubleshooting

Portal or SSH cannot be reached

Confirm Tailscale is running, the invited identity is active and the shared machine appears in `tailscale status`.

Permission denied (publickey)

Check the SSH username, `IdentityFile`, Portal key and provisioning status. Allow about one minute after adding or revoking a key. `ssh -v z6` shows which public identity was offered; never share the private key contents.

CUDA is unavailable

Confirm the script requested a GPU, runs through Slurm and activates the intended environment. Inspect `CUDA_VISIBLE_DEVICES`. Running on the login VM cannot expose a GPU and is not a workaround.

Memory failure

Distinguish host-memory failure from CUDA device-memory failure. Increase `--mem` for the former; reduce model/batch size or use an appropriate distributed strategy for the latter.

Disk quota exceeded

Inspect both home and project use, remove disposable files and contact the administrator only if the approved research requirement has changed.

14. Security and acceptable use

Accounts are personal. Do not share credentials, private keys or Tailscale invitations; expose services; bypass Slurm; scan networks; or interfere with other users. Sensitive, personal, licensed or export-controlled data requires prior approval and applicable TU Delft/project controls.

The full public policy is ACCEPTABLE_USE.md. Administrators may cancel jobs or suspend access to protect the service and other users.

15. Support checklist

Before contacting `b.y.ding@tudelft.nl`, collect:

1. the affected job ID;
2. the exact command or batch-script path;
3. the relevant output/error log path;
4. the observed and expected result; and
5. the approximate time of failure.

Never include passwords, private keys, access tokens or sensitive dataset contents.

Appendix A — Runbook sequence

1. Account application
2. Portal activation
3. SSH key and access
4. Storage and transfer
5. Python environment
6. Interactive allocation
7. Single-GPU job
8. Two-GPU job
9. Job arrays
10. Monitoring and troubleshooting

Appendix B — Command summary

```
ssh z6
module load 2026
sinfo
squeue -u "$USER"
gpustat
gpustat -i 2
sbatch JOB.sbatch
scontrol show job JOB_ID
scancel JOB_ID
df -h "$HOME" /projects
du -sh "$HOME" ~/projects/* 2>/dev/null
```